VIT VELLORE

Vellore Institute of Technology

ADDRESSING MODES

IN INSTRUCTION SET ARCHITECTURE

Scenarios, Diagrams, Comparative Analysis & ISS Design Guide

Student Name	**Aryan Singh Sikarwar**
Registration No.	24BCE0403
Institution	Vellore Institute of Technology, Vellore
Subject	Computer Architecture & Organization (CAO)
Submission Type	Case Study
Date	April 2026

Computer Architecture & Organization · CAO Case Study · VIT Vellore

Table of Contents

1. Introduction

1.1 Abstract

This case study presents a comprehensive analysis of addressing modes in the context of designing a new Instruction Set Architecture (ISS) for the Computer Architecture and Organization (CAO) curriculum at VIT Vellore. Twelve distinct addressing modes are examined through eleven real-world design scenarios, ten annotated diagrams, and a comparative study of RISC versus CISC addressing philosophies. The study covers effective address computation, instruction encoding trade-offs, pipeline interaction, performance implications, security considerations, and practical guidelines for selecting the optimal addressing mode in each scenario. Real-world ISAs — RISC-V, ARM AArch64, and x86-64 — are used as case references throughout.

1.2 What is an Addressing Mode?

An **addressing mode** is a mechanism defined within an instruction that specifies how the processor locates the operand needed for execution. It answers the question: *"where is the data?"* — whether in the instruction itself, in a register, in memory at a fixed address, or at an address computed from register values and constants.

Consider a simple arithmetic instruction: **ADD R1, R2, #5**. This single instruction uses three addressing modes simultaneously: R1 (destination) uses **register addressing**; R2 (source) also uses register addressing; and the constant 5 uses **immediate addressing**. The ISS designer must specify which mode combinations are legal for each instruction class and implement corresponding hardware for each.

In the context of the CAO curriculum, understanding addressing modes is critical because they bridge the gap between high-level programming constructs and low-level machine execution. Every array access, pointer dereference, function call, and conditional branch ultimately relies on one or more addressing modes to locate its operand.

Why Addressing Modes Matter — CAO Perspective
Instruction Encoding: More modes require more opcode/mode bits → affects instruction word width
Pipeline Performance: Each memory indirection adds latency cycles to the execute/memory stages
Compiler Design: Compilers generate better code when the ISS supports modes that match language constructs
Hardware Cost: Complex modes (scaled indexed) require multipliers in the AGU, increasing chip area
Code Density: Richer modes allow more work per instruction → smaller binary footprint
Security: PC-relative mode enables ASLR; indirect modes create attack surfaces if unconstrained

1.3 Scope of this Case Study

This case study covers the following topics in depth, structured to align with VIT Vellore's CAO course objectives:

1. Formal definition of effective address (EA) computation for all twelve modes
2. Hardware context: AGU, register file, pipeline stage interactions
3. Eleven design scenarios showing which mode to choose and why
4. Ten annotated diagrams illustrating mode mechanics at the hardware level
5. Instruction encoding trade-offs in fixed-width (RISC) vs variable-length (CISC) ISAs
6. Performance analysis: cycles, cache behavior, and code density comparison
7. Security implications: ASLR, stack smashing, Spectre side-channels
8. Real-world ISA case studies: RISC-V, ARM AArch64, x86-64
9. Recommended addressing mode set for a new ISS with justification

1.4 Effective Address — Formal Definition

For every addressing mode, the processor hardware computes an **Effective Address (EA)** — the final byte address in the memory space from which the operand is read or to which it is written. The method of computing the EA *is* the defining characteristic that distinguishes one mode from another.

Mode	EA Formula	Extra Mem Accesses	Typical Pipeline Cost
Immediate	No EA (value in instruction)	0	0 extra cycles
Register	No EA (value in register file)	0	0 extra cycles
Direct	EA = Addr field	1	~4 cy (L1 hit)
Indirect	EA = Mem[Addr field]	2	~8 cy (L1 hits)
Register Indir	EA = Reg[Rn]	1	~4 cy (L1 hit)
Base+Offset	EA = Reg[Rb] + offset	1	AGU+~4 cy
Indexed	EA = Reg[Rb] + Reg[Ri]	1	AGU+~4 cy
PC-Relative	EA = PC + signed_offset	0 (branch), 1 (load)	AGU only (branch)
Stack	EA = SP (±size auto)	1	~4 cy (usually L1)
Auto-Increment	EA = Reg; Reg += size	1	AGU+writeback
Scaled Indexed	EA = Rb + Ri×Scale + disp	1	AGU (mult-add)+~4 cy

Mode	EA Formula	Extra Mem Accesses	Typical Pipeline Cost
Auto-Decrement	Reg -= size; EA = Reg	1	AGU+writeback

2. CPU & Memory Architecture Context

Before analyzing individual modes, it is essential to understand the hardware context in which address generation occurs. Every addressing mode computation happens inside the processor's **Address Generation Unit (AGU)** during the Execute stage of the pipeline. The diagram below shows a simplified processor with all the relevant components.

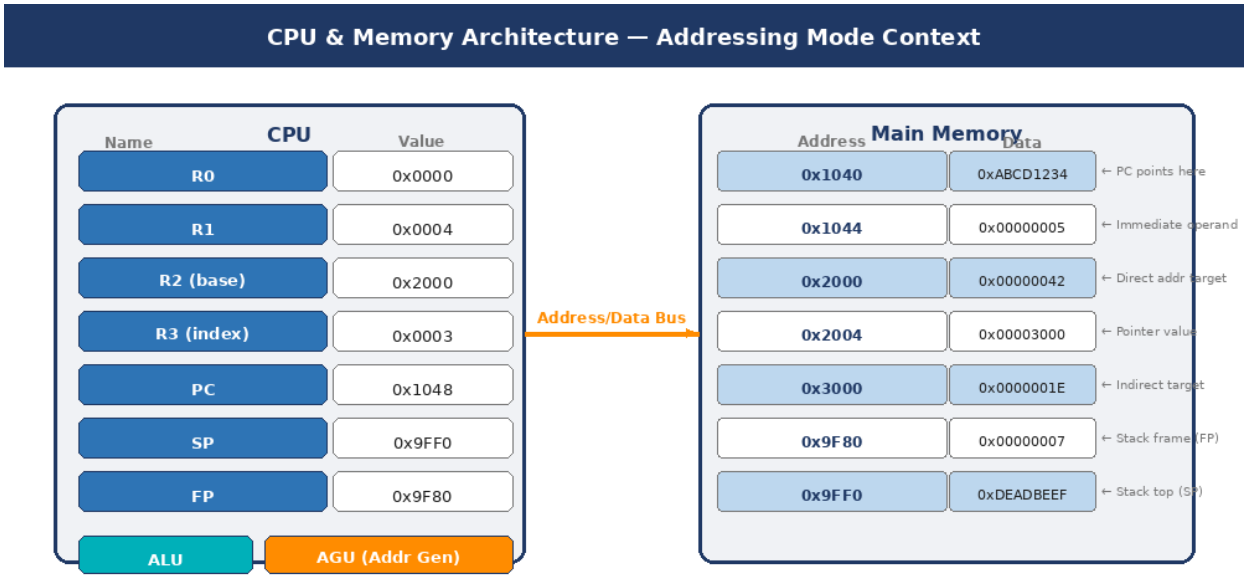

Figure 2.1 — Simplified CPU with Register File (R0–R3, PC, SP, FP), ALU, AGU, and Main Memory. The orange bus carries the effective address computed by the AGU to the memory subsystem. Different addressing modes use different inputs to the AGU.

In this architecture, the register file holds eight visible registers: R0–R3 (general purpose), the Program Counter (PC), Stack Pointer (SP), and Frame Pointer (FP). The **AGU** sits adjacent to the ALU and receives its inputs from the instruction's immediate field and the register file outputs. Its output — the effective address — is forwarded to the memory subsystem (L1 cache → L2 → DRAM).

Hardware Components and Their Role in Addressing
Register File — Holds base pointers (Rb), index values (Ri), and general-purpose operands
Program Counter (PC) — Source value for PC-Relative branches; automatically incremented after each fetch
Stack Pointer (SP) — Implicitly modified by PUSH/POP; source/destination for stack addressing
Frame Pointer (FP) — Stable base for local variable access within a function's stack frame
AGU (Address Generation Unit) — Dedicated hardware adder (and optionally multiplier) for EA computation

Instruction Register (IR) — Holds the fetched instruction; source of opcode, register fields, immediates
L1/L2 Cache — Typically 4–12 cycle hit latency; miss causes 50–200 cycle DRAM penalty

2.1 Five-Stage Pipeline and Addressing Mode Interaction

In a classic five-stage RISC pipeline, each addressing mode engages different pipeline stages. Understanding this mapping explains why some modes are one-cycle and others introduce structural hazards:

Pipeline Stage	What happens for addressing	Modes primarily involved
IF — Instruction Fetch	PC sent to I-cache; instruction retrieved; PC incremented	All modes (PC-Rel uses current PC)
ID — Instruction Decode	Register numbers extracted; register file read; immediate sign-extended	Immediate, Register
EX — Execute (AGU)	EA computed: Rb+offset, Rb+Ri, PC+offset, SP±, Rb+Ri×S+D	All memory modes
MEM — Memory Access	EA sent to D-cache; data read or written	Direct, Indirect, Base+Offset, Stack, Auto
WB — Write Back	Result written to destination register; SP/Ri updated (post-increment)	Auto-Increment, Auto-Decrement

2.2 Data Hazards Caused by Addressing Modes

Some addressing modes introduce **data hazards** in pipelined processors. The most common is the **load-use hazard**: when a LOAD instruction is immediately followed by an instruction that uses the loaded value, the pipeline must insert a one-cycle stall (or the compiler must schedule a NOP).

```
; Load-use hazard example (Base+Offset mode)
LW    x1, 0(x5)      ; load from memory → x1 available after WB
ADD   x2, x1, x3     ; x1 NOT YET READY — 1-cycle stall inserted

; Compiler fix: schedule independent instruction between load and use
LW    x1, 0(x5)      ; load
ADD   x3, x4, x6     ; independent — keeps pipeline busy
ADD   x2, x1, x3     ; x1 now ready — no stall
```

3. Twelve Addressing Modes — Visual Overview

The twelve addressing modes can be organized into three families based on the source of the effective address. The color-coded grid below presents all twelve modes with complexity ratings and primary use cases. Green = low hardware cost; Orange = moderate; Red = high AGU complexity.

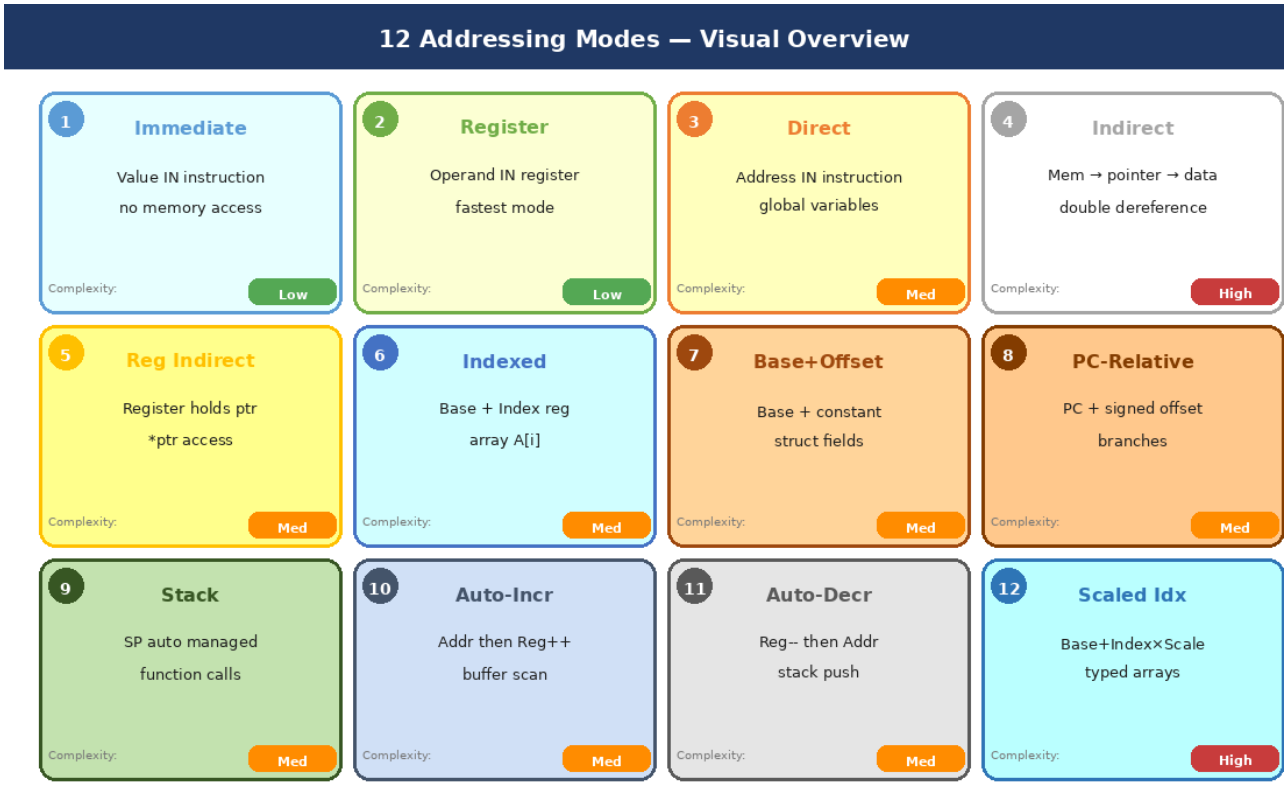

Figure 3.1 — Twelve addressing modes organized by category. Badge numbers correspond to scenario numbers in Section 5. The complexity tag in each card's bottom-right corner reflects AGU hardware cost.

These twelve modes divide naturally into three families:

Family	Modes	EA Source	Key Characteristic
No-Address Family	Immediate, Register	Instruction bits or Register	Zero memory accesses — fastest
Single-Indirect Family	Direct, Reg Indirect, Base+Offset, Indexed, PC-Rel, Stack	AGU computation	One memory access; varies by AGU complexity
Side-Effect Family	Auto-Increment, Auto-Decrement, Scaled Indexed, Indirect	AGU + register writeback	Post-execution register modification or double-deref

3.1 Mode Classification by ISA Philosophy

Not all ISAs support all twelve modes. The choice of which modes to include is one of the defining decisions that separates RISC from CISC architectures. The following table shows which modes appear in the three most important modern ISAs:

Mode	RISC-V	ARM AArch64	x86-64	Recommended for new ISS?
Immediate	✓ I-type	✓	✓	Yes — essential
Register	✓ R-type	✓	✓	Yes — essential
Base+Offset	✓ (12-bit)	✓ (9/12-bit)	✓ (8/32-bit disp)	Yes — essential
PC-Relative	✓ B/J-type	✓ (branches+ADRP)	✓ (RIP-relative)	Yes — essential
Register Indirect	✓ (0-offset)	✓	✓	Yes — via zero-offset Base+Offset
Indexed	✗	✓ (Rb+Ri)	✓ (Rb+Ri)	Optional
Scaled Indexed	✗	✗	✓ (Rb+Ri×S+D)	Optional (high cost)
Auto-Increment	✗	✓ (pre/post)	✗ (only STR/LDR)	Optional (embedded)
Auto-Decrement	✗	✓ (pre-index)	✗	Optional (embedded)
Stack (implicit)	Via SW/LW	PUSH/POP equiv	PUSH/POP	Optional
Direct (absolute)	✗ (use LUI+LW)	✗	✓ (64-bit disp)	No — too wide
Indirect	✗	✗	✗	No — too slow

4. Instruction Encoding and Mode Representation

4.1 Fixed-Width Encoding (RISC Approach)

In fixed-width ISAs like RISC-V (all instructions are exactly 32 bits), the available bits must be shared among the opcode, register specifiers, and the immediate/offset constant. This creates a fundamental tension: wider immediates mean more addressing flexibility, but fewer bits remain for other fields.

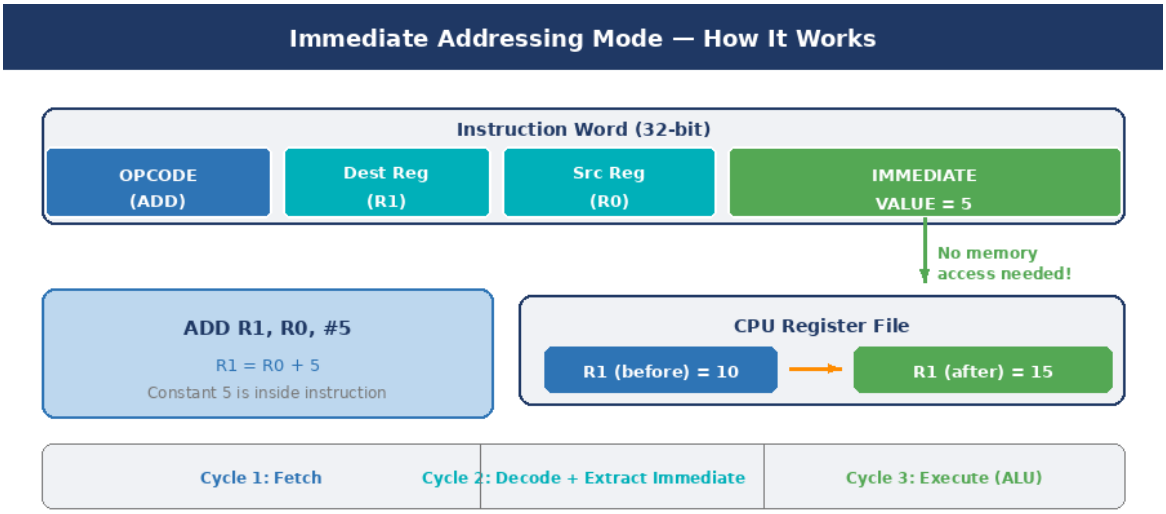

Figure 4.1 — RISC-V I-type instruction encoding: 7-bit opcode, 5-bit destination register, 3-bit funct3, 5-bit source register, and 12-bit signed immediate. The immediate value (here: +5) is embedded in bits [31:20] and sign-extended to 32/64 bits.

RISC-V defines six instruction formats, each optimized for a different addressing scenario:

Format	Opcode Bits	Register Fields	Immediate Bits	Primary Use
R-type	7	rd(5)+rs1(5)+rs2(5)	None (5-bit funct)	Register mode: ALU ops
I-type	7	rd(5)+rs1(5)	12-bit signed	Immediate + Base+Offset loads
S-type	7	rs1(5)+rs2(5)	12-bit split	Base+Offset stores
B-type	7	rs1(5)+rs2(5)	13-bit PC-offset	PC-Relative branches
U-type	7	rd(5)	20-bit upper	Large immediate (LUI)
J-type	7	rd(5)	21-bit PC-offset	PC-Relative long jumps (JAL)

4.2 Variable-Length Encoding (CISC Approach)

x86-64 uses variable-length instructions (1–15 bytes) and encodes addressing modes in a dedicated **ModR/M byte** that follows the opcode. An optional **SIB (Scale-Index-Base) byte** extends this for scaled indexed addressing. This scheme allows great flexibility but makes the decoder significantly more complex.

```
; x86-64 instruction encoding structure:
; [Legacy Prefix] [REX Prefix] [Opcode] [ModR/M] [SIB] [Displacement]
[Immediate]

; ModR/M byte: bits [7:6]=Mod, [5:3]=Reg, [2:0]=R/M
;   Mod=11: register mode  (no memory access)
;   Mod=00: indirect mode  (register holds address)
;   Mod=01: disp8 mode     (register + 8-bit offset)
;   Mod=10: disp32 mode    (register + 32-bit offset)

; SIB byte: bits [7:6]=Scale, [5:3]=Index, [2:0]=Base
;   Scale: 00=×1, 01=×2, 10=×4, 11=×8

MOV EAX, EBX           ; 2 bytes:  opcode + ModR/M(Mod=11)
MOV EAX, [EBX]         ; 2 bytes:  opcode + ModR/M(Mod=00)
MOV EAX, [EBX+8]       ; 3 bytes:  opcode + ModR/M(Mod=01) + disp8
MOV EAX, [EBX+ECX*4]   ; 3 bytes:  opcode + ModR/M + SIB
MOV EAX, [EBX+ECX*4+8] ; 4 bytes:  opcode + ModR/M + SIB + disp8
```

4.3 Immediate Size Limitations and Workarounds

A critical constraint in any fixed-width ISS is that the immediate field size is limited. In RISC-V, the 12-bit I-type immediate can represent values from −2048 to +2047. For larger constants or far-away addresses, a two-instruction sequence is required:

```
; Loading a 32-bit constant 0xDEAD_BEEF into x1 (RISC-V)
LUI   x1, 0xDEADB      ; x1 = 0xDEADB000 (upper 20 bits)
ADDI  x1, x1, 0xEEF    ; x1 = 0xDEADB000 + 0xEEF = 0xDEADBEEF

; Note: if lower 12 bits have bit[11]=1, must adjust LUI by +1
; because ADDI sign-extends — the assembler handles this automatically

; ARM equivalent (AArch64) — MOVZ + MOVK
MOVZ  X1, #0xBEEF, LSL #0     ; X1 = 0x000000000000BEEF
MOVK  X1, #0xDEAD, LSL #16    ; X1 = 0x00000000DEADBEEF
```

5. Eleven Design Scenarios for Choosing Addressing Modes

This section presents eleven concrete programming and hardware design scenarios. For each scenario, the appropriate addressing mode is identified, explained with hardware-level reasoning, illustrated with assembly code, and evaluated for trade-offs. These scenarios together cover the full range of situations a processor designer or assembly programmer encounters.

Scenario 1 — Constants and Literals (Immediate Addressing)

Mode Selected: Immediate Addressing | Hardware Cost: Lowest

Immediate addressing embeds the operand value directly in the instruction binary. The processor extracts the value from the Instruction Register (IR) during the Decode stage — no memory access, no register lookup, no AGU computation. This makes it the fastest possible mode for supplying a constant operand.

Scenarios requiring Immediate Addressing:

- **MOV R1, #0 — start variable at a compile-time constant:** Loop counter initialization:
- Fixed arithmetic: adding 1 to a counter, subtracting a stride, multiplying by a small power of 2
- Bit manipulation: applying masks (AND R, R, #0xFF), setting bits (ORR R, R, #0x80), clearing bits
- Comparisons against thresholds: CMP R3, #100 — branch if counter exceeds 100
- Loading small constants into registers as the first step of an address calculation

Assembly Examples:

```
; ARM / RISC-V Immediate Addressing Examples
MOV  R1, #0            ; R1 = 0              (initialize counter)
ADD  R1, R1, #1        ; R1 = R1 + 1        (increment — no memory)
AND  R2, R2, #0x0F     ; R2 = lower nibble   (bit mask constant)
ORR  R3, R3, #0x80     ; set bit 7 of R3
CMP  R4, #255          ; compare R4 with 255

; RISC-V I-type (12-bit signed immediate)
addi x1, x0, 42       ; x1 = 0 + 42 = 42
ori  x2, x2, 0xFF     ; x2 = x2 | 0xFF
slti x3, x1, 100      ; x3 = (x1 < 100) ? 1 : 0
```

Immediate Mode — Design Constraints
Range in RISC-V (12-bit signed): −2048 to +2047
Range in ARM AArch64: 8-bit value with 4-bit rotation field (modified immediate encoding)

For values outside range: use LUI+ADDI (RISC-V) or MOVZ+MOVK (ARM) two-instruction sequences
Tradeoff: Widening the immediate field (e.g., 16-bit) reduces bits available for registers/opcode
Best practice: Limit immediates to 12 bits; provide a separate 'load upper immediate' instruction

Scenario 2 — High-Speed Computation (Register Addressing)

Mode Selected: Register Addressing | Hardware Cost: Lowest

Register addressing is the foundation of RISC architectures. When both source operands and the destination are CPU registers, the processor accesses the on-chip register file (read latency: typically 1–2 cycles) without any memory bus traffic. This makes register addressing the highest-bandwidth, lowest-latency addressing mode available.

Scenarios requiring Register Addressing:

- All ALU operations between in-flight values: intermediate results of complex expressions
- Loop-carried variables: counters, accumulators, running sums or products
- Pointer arithmetic before a memory access: computing a new address from an existing one
- Boolean flag computations: result of comparison instructions stored in a register for conditional use

```
; All register addressing — zero memory accesses
ADD   R3, R1, R2        ; R3 = R1 + R2
MUL   R4, R3, R5        ; R4 = R3 * R5
LSL   R6, R4, #2        ; R6 = R4 << 2 (×4)  — immediate shift, still
register dst
XOR   R7, R7, R7        ; R7 = 0 (clear register efficiently)
MOV   R8, R4            ; R8 = R4 (register copy)

; RISC-V register file: 32 registers x0..x31
add   x3, x1, x2        ; R-type: all operands in register file
sub   x4, x3, x5
and   x6, x4, x7
sll   x8, x6, x9        ; shift left logical by amount in x9
```

Scenario 3 — Global Variables and Memory-Mapped I/O (Direct Addressing)

Mode Selected: Direct (Absolute) Addressing | Hardware Cost: Medium

Direct addressing places the complete memory address inside the instruction word. The EA equals the address field exactly — one memory access, no AGU arithmetic. This mode is appropriate for global/static variables whose addresses are fixed at link time and for hardware peripheral registers at known physical addresses in embedded systems.

```
; Direct addressing — address in instruction
LOAD  R1, 0x4000       ; R1 = Mem[0x4000]  (read global variable)
STORE R2, 0x4004       ; Mem[0x4004] = R2  (write global variable)

; Memory-mapped I/O — embedded system (ARM Cortex-M)
#define GPIOA_IDR  0x40010808   ; GPIO input data register address
#define UART_TDR   0x40011004   ; UART transmit data register
LOAD  R0, =GPIOA_IDR   ; load address constant into R0
LOAD  R1, [R0]         ; read GPIO — actually Register Indirect!

; Why RISC-V has NO true Direct mode:
; A 32-bit address cannot fit in a 32-bit instruction alongside opcode+rd
; RISC-V workaround for global variable at 0x20004000:
lui   x1, 0x20004      ; x1 = 0x20004000 (upper 20 bits)
lw    x2, 0(x1)        ; load from that address (register indirect,
offset=0)
```

Note that pure RISC architectures like RISC-V **do not include** a true direct addressing mode. The instruction width constraint makes it impossible to fit a full 32-bit address alongside the opcode and destination register. The two-instruction LUI+LW sequence is the standard substitute.

Scenario 4 — Pointers, Arrays, and Struct Fields

Mode Selected: Register Indirect + Base+Offset | Hardware Cost: Medium

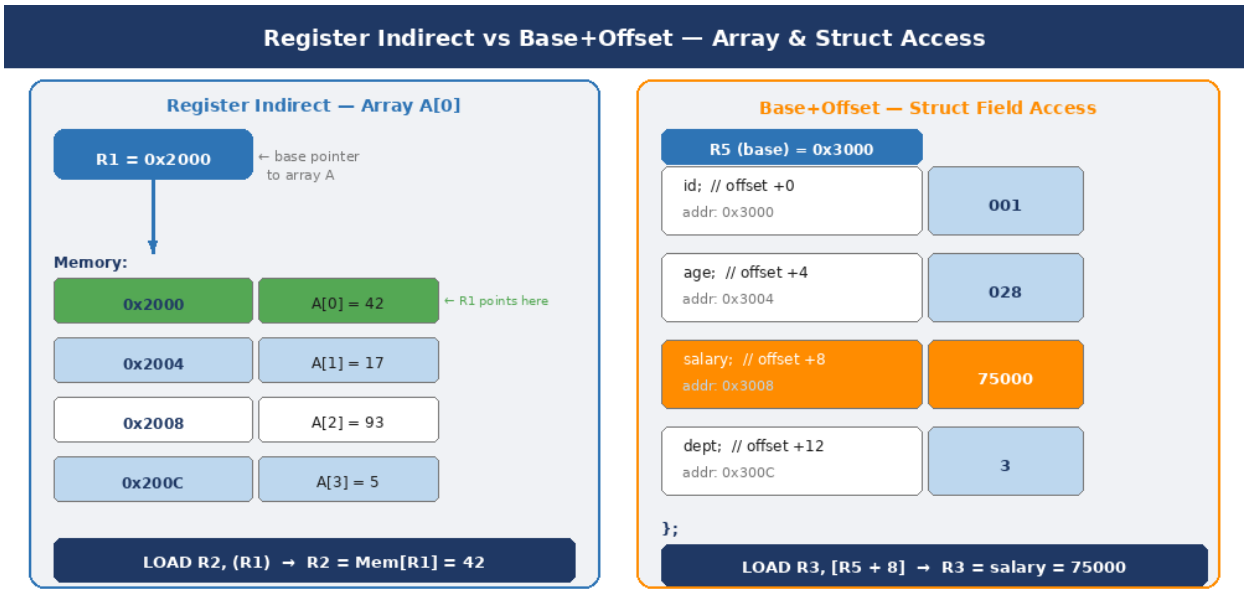

Figure 5.1 — Left: Register Indirect for pointer dereference (LOAD R2, (R1) reads from the address stored in R1). Right: Base+Offset for struct field access — the 'salary' field at byte offset +8 from the struct base pointer in R5.

These two closely related modes are the most frequently used memory-access modes in real programs. **Register Indirect** is the special case of Base+Offset where the offset is zero — the register alone provides the address. **Base+Offset** adds a compile-time constant displacement to the register, enabling access to fields within a data structure.

```
; Register Indirect: pointer dereference   EA = Reg[R1]
LOAD  R2, (R1)            ; R2 = *ptr  (dereference pointer in R1)
STORE R3, (R1)            ; *ptr = R3

; Base+Offset: struct field access   EA = Reg[R5] + constant
; struct Employee { int id; int age; int salary; int dept; };
; offsets: id=0, age=4, salary=8, dept=12  (4 bytes each)
LOAD  R0, [R5 + 0]        ; R0 = emp->id
LOAD  R1, [R5 + 4]        ; R1 = emp->age
LOAD  R2, [R5 + 8]        ; R2 = emp->salary  ← most common pattern
LOAD  R3, [R5 + 12]       ; R3 = emp->dept

; RISC-V: lw rd, offset(rs1)
lw    x10, 8(x5)          ; load salary field  (offset=8 bytes)
sw    x11, 8(x5)          ; store salary field
```

Scenario 5 — Variable-Index Array Traversal (Indexed Addressing)

Mode Selected: Indexed Addressing | **Hardware Cost: Medium**

Indexed addressing computes EA = Base_Register + Index_Register, where both registers can hold independently varying values. This two-register form is ideal for nested loops (matrix access), two-pointer algorithms, and any scenario where both the base and the index change dynamically during execution.

```
; Indexed addressing — two-register EA computation
; int A[N], B[N];  compute dot product: sum += A[i]*B[i]
MOV   R1, addr_A       ; R1 = base of array A
MOV   R2, addr_B       ; R2 = base of array B
MOV   R3, #0           ; R3 = index (byte offset, starts at 0)
MOV   R4, #0           ; R4 = accumulator
loop:
  LOAD  R5, [R1+R3]    ; R5 = A[i]  — indexed: base A + index
  LOAD  R6, [R2+R3]    ; R6 = B[i]  — indexed: base B + index
  MUL   R7, R5, R6     ; R7 = A[i] * B[i]
  ADD   R4, R4, R7     ; sum += product
  ADD   R3, R3, #4     ; advance index by sizeof(int)=4
  CMP   R3, #40        ; done when index reaches N*4=40
  BLT   loop
```

Scenario 6 — Branches and Position-Independent Code (PC-Relative)

Mode Selected: PC-Relative Addressing | Hardware Cost: Low (AGU adder only)

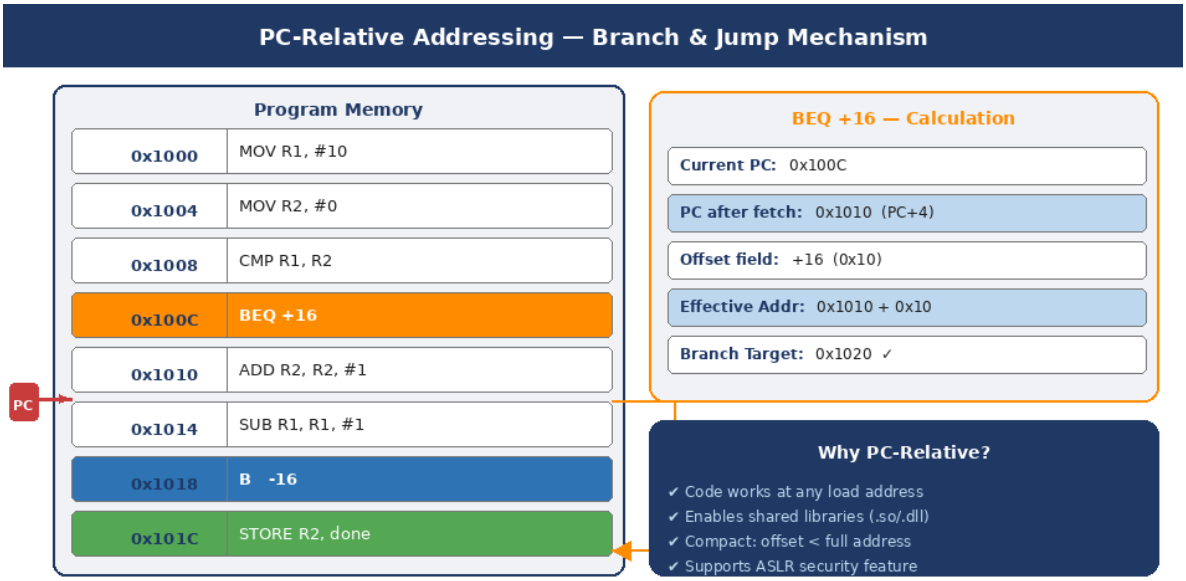

Figure 5.2 — PC-Relative branch mechanism. The BEQ instruction at 0x100C adds its signed offset (+16) to PC+4 (0x1010) to produce target 0x1020. The backward branch at 0x1018 uses a negative offset to return to the loop start.

PC-relative addressing computes the branch target as the sum of the current (or next-instruction) Program Counter and a signed offset embedded in the instruction. This mode is mandatory for branches and is the key enabler of **Position-Independent Code (PIC)**, which can be loaded at any memory address and execute correctly.

```
; PC-Relative branch examples
BEQ  R1, R2, +16       ; if R1==R2: PC = PC+4+16 = jump forward
BNE  R1, R0, -20       ; if R1!=0:  PC = PC+4-20 = jump backward (loop)

; RISC-V B-type (13-bit signed offset, byte-addressed)
beq   x1, x2, target   ; assembler computes offset automatically
bne   x1, x0, loop_start
jal   x1, function     ; call function: x1=return addr, PC=PC+offset

; ARM AArch64 — PC-relative data access (ADRP + LDR)
ADRP  X0, mydata@PAGE  ; X0 = page base of mydata (PC-relative, ±4GB range)
LDR   X1, [X0, mydata@PAGEOFF]  ; load from computed address

; Why PC-Relative enables ASLR:
; If the whole program is loaded 0x10000 bytes higher,
; every PC-relative offset still points to the correct target
; because both PC and target shifted by the same amount.
```

Scenario 7 — Function Calls and Recursion (Stack Addressing)

Mode Selected: Stack Addressing + Base+Offset (FP-based) | Hardware Cost: Low-Medium

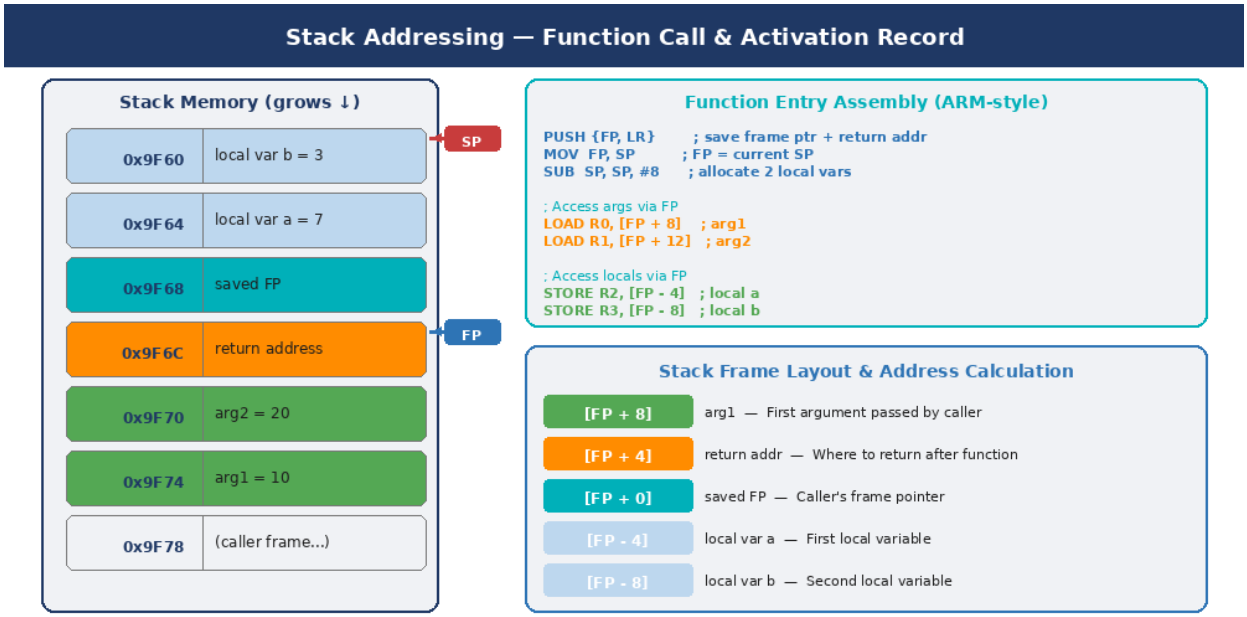

Figure 5.3 — Activation record (stack frame) for a function call. The SP descends as the frame is built. FP provides a stable reference — arguments are at positive offsets above FP, locals at negative offsets below FP.

The call/return mechanism in virtually all modern ISAs relies on the stack. The Stack Pointer (SP) is automatically decremented on push and incremented on pop, providing an implicit addressing mode. Within the frame, Base+Offset with the Frame Pointer (FP) provides stable access to both arguments and local variables regardless of how deep inner calls go.

```
; Complete function prologue/epilogue (ARM AArch32 style)

; --- ENTRY: called function saves state ---
PUSH  {R4-R11, LR}     ; save callee-saved regs + link reg; SP -= 36
MOV   FP, SP           ; FP = current SP (frame pointer established)
SUB   SP, SP, #16      ; allocate 4 local ints (4 bytes each); SP -= 16

; --- ACCESS: use Base+Offset relative to FP ---
LOAD  R0, [FP + 36]    ; arg1 (above saved registers)
LOAD  R1, [FP + 40]    ; arg2
STORE R2, [FP - 4]     ; local variable a
STORE R3, [FP - 8]     ; local variable b

; --- RECURSION: inner call gets own frame below current SP ---
BL    factorial         ; recursive call; SP untouched until prologue
```

```
; --- EXIT: restore and return ---
MOV   SP, FP            ; deallocate locals
POP   {R4-R11, PC}      ; restore regs; LR→PC = return to caller
```

Scenario 8 — Sequential Buffer Operations (Auto-Increment)

Mode Selected: Auto-Increment / Auto-Decrement | **Hardware Cost: Low-Medium**

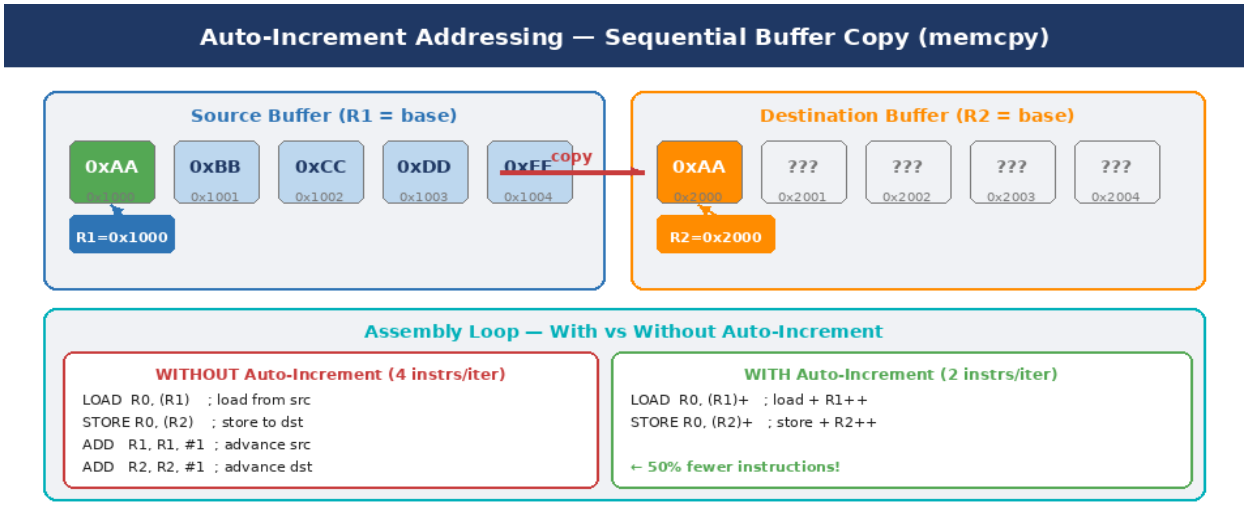

Figure 5.4 — Auto-increment addressing eliminates the explicit ADD-to-pointer instruction. Left buffer (src, R1): green cell is the currently accessed element, R1 advances automatically. Right buffer (dst, R2): first element already copied.

Auto-increment addressing performs the memory access at the current register value and then automatically increments the register by the element size. This is purpose-built for sequential buffer traversal — character string scanning, memcpy operations, and DMA buffer filling — eliminating the explicit pointer-advance instruction that would otherwise be required after every access.

```
; Auto-increment vs manual increment — byte-copy loop

; WITHOUT auto-increment: 4 instructions per iteration
loop_manual:
  LOAD  R0, (R1)          ; R0 = Mem[R1]
  STORE R0, (R2)          ; Mem[R2] = R0
  ADD   R1, R1, #1        ; advance src pointer manually
  ADD   R2, R2, #1        ; advance dst pointer manually
  SUB   R3, R3, #1
  BNE   R3, #0, loop_manual

; WITH auto-increment: 2 instructions per iteration (50% fewer)
loop_auto:
  LOAD  R0, (R1)+         ; R0 = Mem[R1]; R1++  (post-increment)
  STORE R0, (R2)+         ; Mem[R2] = R0; R2++
  SUB   R3, R3, #1
```

```
  BNE   R3, #0, loop_auto

; ARM LDMIA — extreme auto-increment (8 regs at once)
LDMIA R1!, {R4-R11}     ; load 8 words (32 bytes); R1 += 32
STMIA R2!, {R4-R11}     ; store 8 words; R2 += 32
; Copies 32 bytes in 2 instructions!
```

Scenario 9 — Function Pointers and Virtual Dispatch (Register Indirect Jump)

Mode Selected: Register Indirect (for CALL/JMP) | **Hardware Cost: Low**

When the target address of a call or jump is not known until runtime — as in C++ virtual function dispatch, callback function invocation, or dynamic library symbol resolution — the address is first loaded into a register, and the branch instruction uses that register as its target via register indirect addressing.

```
; Function pointer call in C → assembly
; void (*callback)(int x) = &my_handler;
; callback(42);
LOAD  R5, [fp_ptr]        ; R5 = address of my_handler
MOV   R0, #42             ; set argument x=42
CALL  (R5)                ; jump to address in R5, save return addr

; C++ virtual method dispatch (vtable mechanism)
; obj->draw();  — obj is a pointer to a derived class object
LOAD  R0, [obj]           ; R0 = vtable pointer (first field of object)
LOAD  R1, [R0 + 8]        ; R1 = draw() entry at vtable[2] (offset 8)
MOV   R2, obj             ; pass 'this' pointer as first argument
CALL  (R1)                ; indirect call through vtable

; RISC-V: JALR (Jump And Link Register)
lw    x5, 0(x10)          ; x5 = vtable pointer
lw    x6, 8(x5)           ; x6 = method address
jalr  x1, x6, 0           ; call: x1=return addr, PC=x6
```

Indirect calls make static branch prediction difficult because the target address is only known at runtime. Modern processors use sophisticated **Indirect Branch Target Predictors (IBTP)** that maintain a history of previous target addresses for each call site to predict the most likely next target.

Scenario 10 — Typed Array Access with Scaling (Scaled Indexed)

Mode Selected: Scaled Indexed Addressing | **Hardware Cost: High (AGU multiplier required)**

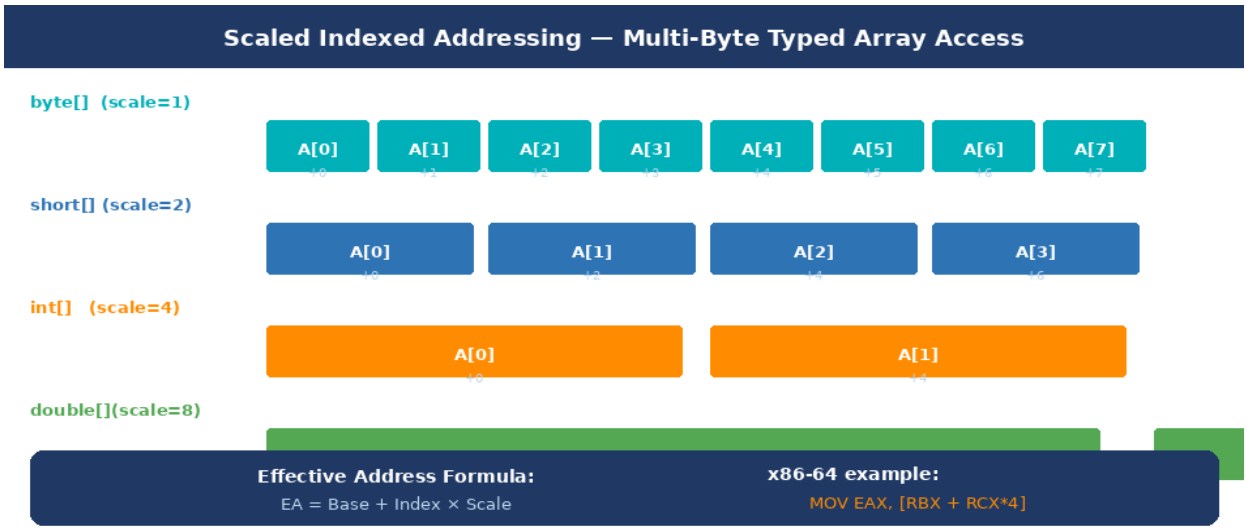

Figure 5.5 — Scaled indexed addressing: the AGU automatically multiplies the index register by the element size (scale factor). byte[]=scale 1, short[]=scale 2, int[]=scale 4, double[]=scale 8. Only x86-64 provides this natively; RISC-V and ARM require manual shift-left.

When iterating over typed arrays using a natural (0-based) index, scaled indexed addressing performs the element-size multiplication inside the AGU hardware: **EA = Base + Index × Scale + Displacement**. Without this mode, the programmer or compiler must manually shift the index left by $\log_2$ (element_size) before each access.

```
; Scaled Indexed — x86-64 (SIB byte provides Scale field)
; int A[8];  for (int i=0; i<8; i++) sum += A[i];
XOR   RCX, RCX                  ; RCX = i = 0
XOR   EAX, EAX                  ; EAX = sum = 0
loop:
  ADD   EAX, [RBX + RCX*4]      ; sum += A[i]; EA = RBX + i×4
  INC   RCX                     ; i++  (not i+=4 — natural index!)
  CMP   RCX, 8
  JL    loop

; Same in RISC-V (no scaled indexed — manual shift required)
addi  x3, x0, 0                 ; i = 0
addi  x4, x0, 0                 ; sum = 0
loop:
  slli  x5, x3, 2               ; x5 = i * 4 (shift left 2 bits)
  add   x5, x10, x5             ; x5 = base_A + i*4
  lw    x6, 0(x5)               ; load A[i]
  add   x4, x4, x6              ; sum += A[i]
  addi  x3, x3, 1               ; i++
  blt   x3, x9, loop
; RISC-V needs 2 extra instructions per iteration vs x86
```

Scenario 11 — Embedded Peripherals and DMA (Combined Modes)

Mode Selected: Direct + Register Indirect + Base+Offset (combined) | Hardware Cost: Varies

Embedded and real-time systems (microcontrollers, SoCs) interact with hardware peripherals through **Memory-Mapped I/O (MMIO)**: peripheral control registers appear at specific physical addresses in the processor's address space. Three addressing strategies are combined in practice:

- Direct addressing for a single peripheral at a compile-time-fixed address
- Register Indirect with a runtime-configurable base for portable driver code
- Base+Offset for accessing multiple registers within a peripheral block

```
; Embedded UART driver — ARM Cortex-M4 (STM32)

; UART register map (offset from USART1 base = 0x40011000):
; +0x00: SR (Status Register)    +0x04: DR (Data Register)
; +0x08: BRR (Baud Rate)         +0x0C: CR1 (Control)

; Strategy A — Direct (compile-time fixed address):
LOAD  R0, =0x40011000   ; load peripheral base
LOAD  R1, [R0, #0]      ; read SR (status)
TST   R1, #0x40         ; test TC (transfer complete) bit

; Strategy B — Register Indirect base (portable across UART0/1/2):
LOAD  R5, [uart_base]   ; R5 = configured UART base (set at runtime)
LOAD  R0, [R5, #0x00]   ; SR — status register
STORE R1, [R5, #0x04]   ; DR — transmit data
LOAD  R2, [R5, #0x08]   ; BRR — baud rate

; To switch from UART1 to UART2, just change uart_base value!
; No recompilation needed — portable driver code.
```

6. Mode Selection Flowchart & Quick-Reference

The following flowchart provides a structured decision procedure for selecting the appropriate addressing mode. It encodes the priority ordering derived from hardware cost analysis: zero-memory-access modes are always preferred when applicable; single-register modes are preferred over dual-register modes; compile-time constants are preferred over runtime-computed addresses.

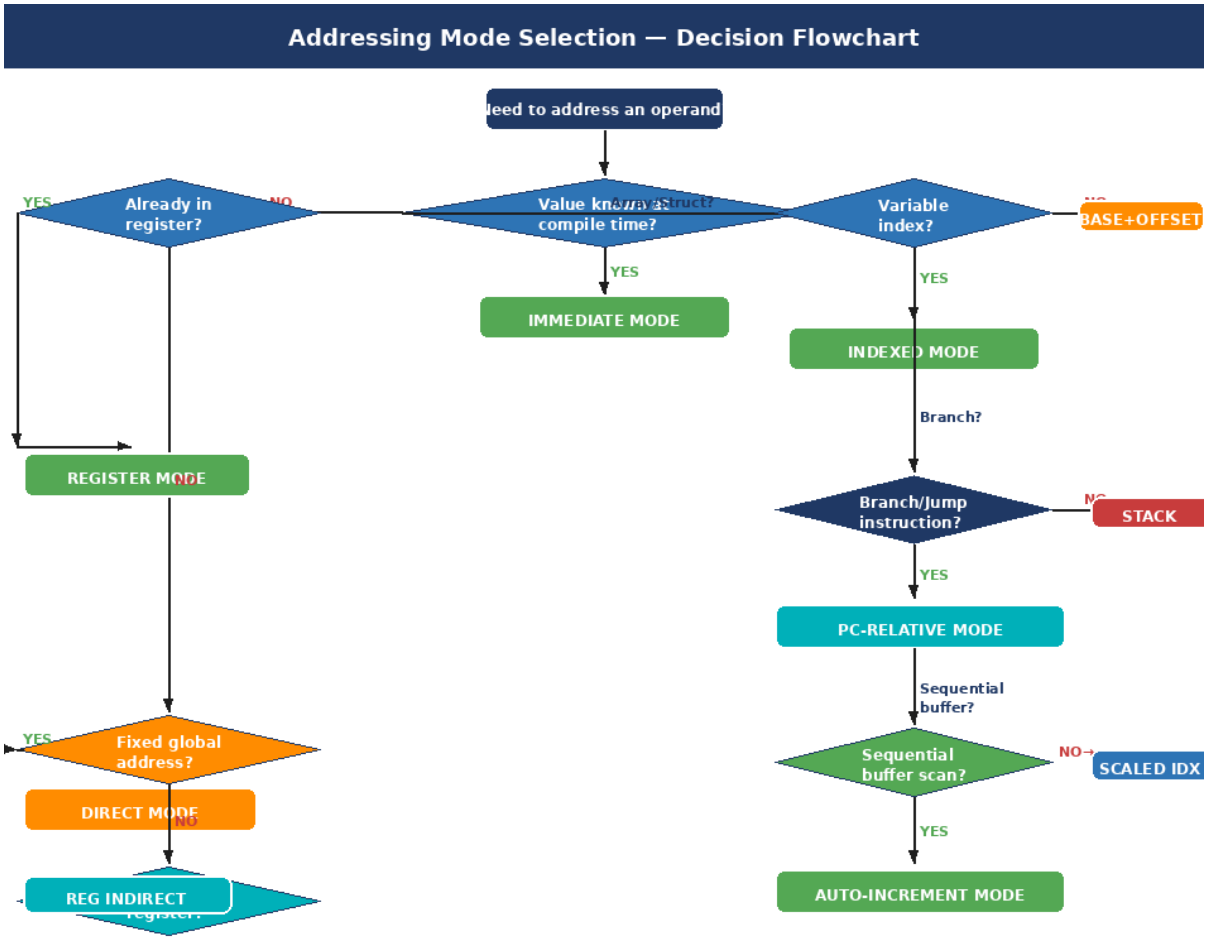

Figure 6.1 — Addressing mode selection decision flowchart. Start at the top and follow Yes/No branches. Green terminal nodes show the recommended mode. This decision tree covers all 12 modes through a series of hardware-grounded questions.

6.1 Quick-Reference Decision Table

Programming Scenario	Best Mode	Reason
Initializing variable to compile-time constant	Immediate	Embedded in instruction — 0 memory cycles
Arithmetic on values already in registers	Register	On-chip register file — fastest access

Programming Scenario	Best Mode	Reason
Reading/writing a global variable (link-time address)	Direct	Fixed address — no computation needed (CISC) or LUI+LW (RISC)
Dereferencing a pointer (*ptr)	Register Indirect	Register holds pointer — 1 memory access
Accessing struct field at fixed offset	Base+Offset	Register+constant — single adder in AGU
Iterating array with variable loop index	Indexed	Two-register EA sum in AGU
Typed array with natural (0-based) integer index	Scaled Indexed	AGU multiplies index by element size
Conditional branch or unconditional jump	PC-Relative	Offset from PC — enables PIC and ASLR
Function call / return / recursion	Stack	Implicit SP management + FP-relative locals
Sequential buffer copy (memcpy-style)	Auto-Increment	Eliminates explicit pointer-advance instruction
Stack push during function call	Auto-Decrement	SP decremented then stored — pre-decrement
Dynamic dispatch (function pointer / vtable)	Reg Indirect (Jump)	Target address only known at runtime

7. RISC vs CISC — Addressing Mode Philosophy

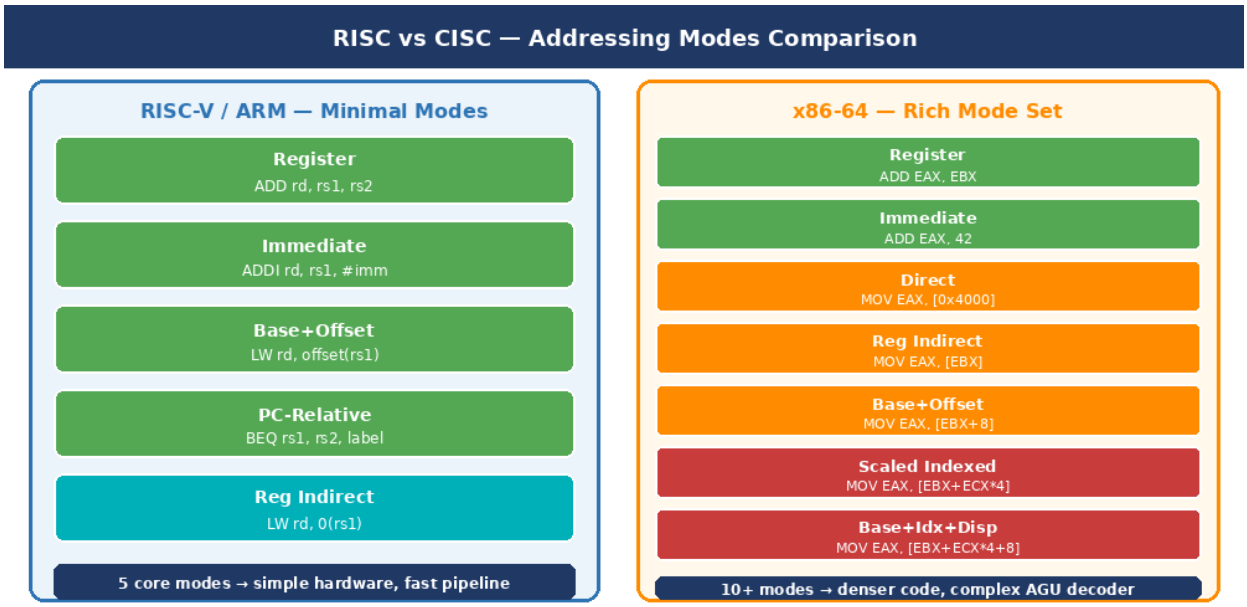

Figure 7.1 — RISC (RISC-V/ARM) mode set (left) versus CISC x86-64 mode set (right). Green modes appear in both; orange/red modes are CISC-specific. RISC cores contain only an adder in the AGU; CISC cores require a multiply-add unit.

The RISC-versus-CISC divide in addressing mode design is not merely academic — it has direct, measurable consequences for transistor count, clock frequency, power consumption, compiler design, and code density. This section analyzes the philosophical and practical differences between the two approaches.

Dimension	RISC (RISC-V, ARM, MIPS)	CISC (x86-64, VAX)
Design philosophy	Simplicity first; let compiler compensate	Expressiveness first; hardware does the work
Mode count	3–5 core modes	10–20+ modes
Instruction width	Fixed 32-bit (or 16+32 Thumb/C)	Variable 1–15 bytes
Mode encoding	Implicit by instruction format	Explicit ModR/M + SIB bytes
AGU hardware	Adder only (+ shift for scaled in CISC)	Multiply-add (for scaled indexed)
Decoder complexity	Simple 1-cycle decode	Multi-cycle, micro-op based
Instructions/operation	More (load-store discipline)	Fewer (memory operands in ALU ops)
Code density	Lower	Higher
Pipeline efficiency	High (predictable, no hidden stalls)	Moderate (variable-length decode stalls)
Power efficiency	High (simple logic, lower transistor count)	Lower (complex decode, more transistors)

Dimension	RISC (RISC-V, ARM, MIPS)	CISC (x86-64, VAX)
Primary use today	Mobile, embedded, HPC (ARM, RISC-V)	Desktop, server (x86-64)

7.1 The Load-Store Principle — A Core RISC Constraint

The load-store principle mandates that only dedicated LOAD and STORE instructions access memory. All other instructions (ADD, SUB, AND, etc.) operate exclusively on registers. As a consequence, RISC processors only need complex addressing logic in the load/store instructions, keeping the instruction decoder and datapath simple and fast.

```
; Task: compute Mem[A] + Mem[B] → Mem[C]

; RISC approach (load-store discipline, RISC-V)
lw   x1, 0(x10)          ; LOAD: A → x1
lw   x2, 0(x11)          ; LOAD: B → x2
add  x3, x1, x2          ; ADD in registers only
sw   x3, 0(x12)          ; STORE: x3 → C
; 4 instructions, all simple, all pipelineable

; CISC approach (x86-64 with memory operands)
MOV  EAX, [RBX]          ; load A
ADD  EAX, [RCX]          ; add B directly from memory!
MOV  [RDX], EAX          ; store to C
; 3 instructions — denser, but ADD has a memory operand
; → more complex AGU, potential decode stall
```

8. Performance Analysis of Addressing Modes

8.1 Latency Profile by Mode

The table below shows the typical access latency profile for each addressing mode on a modern out-of-order processor. Latencies assume a 3 GHz processor with L1 cache hit time of 4 cycles and DRAM latency of 150 ns (~450 cycles):

Mode	AGU Cycles	Mem Accesses	L1 Hit Latency	DRAM Miss Latency	Total (L1 best)
Immediate	0	0	0	N/A	0 cy
Register	0	0	0	N/A	0 cy
Base+Offset	1	1	4	~450	5 cy
Reg Indirect	0	1	4	~450	4 cy
Indexed	1	1	4	~450	5 cy
PC-Relative (branch)	1	0	0	N/A	1 cy
Auto-Increment	1+WB	1	4	~450	6 cy
Scaled Indexed	2	1	4	~450	6 cy
Stack	1	1	4 (usually L1)	~450	5 cy
Indirect	0	2	8	~900	8+ cy
Direct	0	1	4	~450	4 cy

8.2 Code Density Impact

Code density determines how many bytes of memory a program occupies. On systems with small instruction caches (embedded processors, mobile SoCs), denser code means fewer I-cache misses. The following comparison shows the same loop in three ISAs:

```
; Sum of N integers: sum=0; for(i=0;i<N;i++) sum+=A[i]

; RISC-V (32-bit fixed width) — 7 instructions × 4 bytes = 28 bytes
addi  x4, x0, 0        ; sum=0
addi  x3, x0, 0        ; i=0
loop: slli  x5, x3, 2  ; offset = i*4
  add   x5, x10, x5    ; addr = base + offset
  lw    x6, 0(x5)      ; load A[i]
  add   x4, x4, x6     ; sum += A[i]
  addi  x3, x3, 1      ; i++
  blt   x3, x9, loop   ; if i<N goto loop

; ARM Thumb-2 (mixed 16/32-bit) — ~18-20 bytes (denser!)
```

```
MOVS  R4, #0           ; sum=0  (16-bit Thumb)
MOVS  R3, #0           ; i=0
loop: LDR   R6,[R10,R3,LSL#2] ; A[i] w/ scaled indexed (32-bit)
  ADD   R4, R4, R6     ; sum+=
  ADDS  R3, R3, #1     ; i++
  CMP   R3, R9         ; compare
  BLT   loop           ; branch (16-bit Thumb)

; x86-64 — 4 instructions, ~15 bytes
XOR   EAX,EAX          ; sum=0
XOR   ECX,ECX          ; i=0
loop: ADD EAX,[RBX+RCX*4] ; sum+=A[i]  (scaled indexed!)
  INC   ECX
  CMP   ECX,EDX
  JL    loop
```

9. Security Implications of Addressing Mode Choices

9.1 PC-Relative Mode and ASLR

Address Space Layout Randomization (ASLR) is a security mitigation that places the program code, stack, heap, and shared libraries at random virtual addresses each time the program runs. An attacker who exploits a vulnerability cannot rely on knowing where the payload or return addresses are.

ASLR is only possible if the program code uses **PC-relative addressing** exclusively for all internal branches and data references. If any instruction uses an absolute (direct) address, that instruction will fail when the program is loaded at a different base address, breaking ASLR support.

Addressing Mode	ASLR Compatible?	Reason
PC-Relative (branches)	✓ Fully compatible	Target offset fixed; both PC and target shift equally
PC-Relative (data, ADRP)	✓ Fully compatible	Data offset from PC stays valid after relocation
Register Indirect (GOT)	✓ Compatible (with GOT)	Global Offset Table holds runtime-patched addresses
Base+Offset (stack/frame)	✓ Compatible	SP/FP are runtime values, not absolute addresses
Direct (absolute)	✗ Incompatible	Hardcoded addresses break when load address changes
Immediate (code addresses)	✗ Incompatible	Literal address constants break on relocation

This is why RISC-V and ARM AArch64 do not include true direct addressing for code references — it would make Position-Independent Executables (PIE) and shared libraries impossible without complex runtime patching for every absolute reference.

9.2 Stack Addressing and Buffer Overflow Attacks

Stack addressing modes create the primary attack surface for classic **buffer overflow exploits**. When a function allocates a local array on the stack and does not bounds-check writes to it, an attacker can overwrite the saved return address (which lies adjacent on the stack) with a value pointing to malicious code.

Stack Protection Mechanisms — Modern Defenses

Stack Canary: A random value placed between local variables and the saved return address.
→ The runtime checks the canary before returning; if corrupted, the process is terminated.
NX/DEP (Non-Executable Stack): The OS marks stack pages as non-executable.
→ Even if the return address is hijacked, injected shellcode cannot run on the stack.
Shadow Stack (Intel CET, RISC-V Zicfiss): A hardware-protected second copy of return addresses.
→ Any mismatch between the shadow stack and the real stack triggers a fault.
RELRO (Relocation Read-Only): After startup, certain memory segments are marked read-only.
→ Prevents overwriting function pointers in the GOT through write exploits.

9.3 Spectre and Indirect Branch Targeting

The **Spectre** class of vulnerabilities (2018) exploits the speculative execution of indirect branches (Register Indirect jump mode). When the CPU speculatively executes along a mispredicted indirect branch target, it may transiently read memory it should not access. Even though the speculative path is squashed, observable cache timing side-effects reveal the secret data.

- Retpoline: A software trampoline that defeats the Return Stack Buffer predictor, preventing speculation past indirect branches
- IBRS/IBPB (Indirect Branch Restricted Speculation / Predictor Barrier): Intel microcode mitigations
- Control Flow Integrity (CFI): Compiler instrumentation that restricts valid targets of indirect branches to a whitelist
- Hardware CFI (ARM BTI, Intel CET ENDBRANCH): ISA extensions that mark legal indirect branch targets in the instruction stream

10. Designing a New ISS — Recommendations for CAO

10.1 Minimum Viable Mode Set

Based on the analysis in this case study, the following five modes constitute a complete, efficient, and secure minimum viable addressing mode set for a new ISS. This set covers every programming scenario analyzed in Section 5 while maintaining simple hardware and fast decode:

Priority	Mode	Instruction Examples	What it covers
1 — Essential	Immediate	ADDI, LUI, SUBI	Constants, literals, small offsets
2 — Essential	Register	ADD, SUB, AND, OR, SHL	All ALU computation, temporaries
3 — Essential	Base+Offset	LW rd, off(rs), SW rs, off(rb)	Arrays, structs, stack frames, locals
4 — Essential	PC-Relative	BEQ, BNE, JAL	All branches, function calls, PIC/ASLR
5 — Essential	Register Indirect	JALR, LW rd, 0(rs)	Function pointers, vtables (zero-offset Base)

These five modes, implemented with a load-store discipline, require only a single-adder AGU, a simple decoder, and a standard register file. A student-design RISC processor (e.g., in Verilog/VHDL for a CAO lab) can be implemented with these five modes in a standard 5-stage pipeline without structural hazards beyond the standard load-use stall.

10.2 Optional Modes — Add Only with Justification

Mode	Add if your target workload includes...	Additional hardware needed
Auto-Increment/Decrement	Embedded/DSP, memcpy-heavy, string processing	Register writeback path; minimal cost
Scaled Indexed	Heavy use of int/double arrays in C/C++/Java	Multiplier in AGU; moderate cost
Stack PUSH/POP	OS-ABI compatibility; full C runtime support	SP auto-decrement/increment; low cost
Indexed (dual-register)	Matrix algorithms, two-pointer patterns	Second register read port; low cost
Direct (absolute)	Bare-metal embedded with fixed MMIO addresses	Wide instruction format; high encoding cost

Mode	Add if your target workload includes...	Additional hardware needed
Indirect (double-deref)	Hardware virtualization, OS page tables	Double memory latency; avoid if possible

10.3 Complete ISS Design Checklist

10. Define instruction word width (16, 32, or variable). This constrains immediate field sizes and mode encoding bits.
11. Choose the minimum 5-mode core set (Section 10.1) as the foundation.
12. Decide load-store discipline: all memory accesses via explicit LOAD/STORE, or allow memory operands in ALU instructions?
13. Specify AGU complexity: adder-only for RISC core; add multiplier only if scaled indexed mode is required.
14. Define branch offset range: B-type (short, local) vs J-type (long, inter-function). Check if ±4 KB is enough for your target programs.
15. Specify stack convention: hardware PUSH/POP instructions, or software-managed (SP modified via ADDI and accessed via Base+Offset)?
16. Define the Application Binary Interface (ABI): calling convention, register roles, stack frame layout, alignment rules.
17. Ensure PC-Relative mode is available for all branches — required for PIE, shared libraries, and ASLR.
18. Plan for shadow stack extension (Zicfiss or equivalent) if security is a design requirement.
19. Benchmark representative workloads BEFORE adding complex modes — measure benefit vs hardware cost penalty.

11. Real-World ISA Case Studies

11.1 RISC-V (2014) — Open Clean-Slate Design

RISC-V was designed at UC Berkeley starting in 2010 and publicly released in 2014. It represents the most modern from-scratch RISC ISA design, free from decades of legacy constraints. Its addressing mode choices directly reflect the lessons of 40 years of RISC research.

Mode	RISC-V Support	Implementation
Immediate	✓ I/U-type	12-bit signed (I-type), 20-bit upper (U-type)
Register	✓ R-type	All ALU instructions; 32 registers x0..x31
Base+Offset	✓ I/S-type	LW/SW with 12-bit signed offset
PC-Relative	✓ B/J-type	BEQ/BNE (13-bit), JAL (21-bit)
Register Indirect	✓ via JALR	JALR rd, rs1, 0 — zero offset Base+Offset
Auto-Increment	✗	Not in base ISA — use explicit ADDI
Scaled Indexed	✗	Not supported — use SLLI + ADD + LW
Direct	✗	Use LUI + LW pair for 32-bit addresses

RISC-V's decision to omit auto-increment and scaled indexed modes keeps the base ISA (RV32I) small enough to be fully specified in a document of under 100 pages. Extensions like V (Vector) add stride-based and gather/scatter addressing for scientific and multimedia workloads.

11.2 ARM AArch64 (2011) — Pragmatic RISC with Practical Extensions

ARM's 64-bit ISA (used in Apple M1/M2/M3, Qualcomm Snapdragon, AWS Graviton, Samsung Exynos) adds several practical modes beyond the pure RISC minimum — motivated by real-world workload analysis from billions of deployed ARM devices.

- Base+Offset with three sub-variants: unsigned 12-bit scaled, signed 9-bit unscaled, and pre/post-index forms
- Pre-index equivalent to Auto-Decrement: LDR X0, [X1, #-8]! (access then update X1)
- Post-index equivalent to Auto-Increment: LDR X0, [X1], #8 (access first, update after)
- ADRP instruction: PC-relative address of a 4 KB page within ±4 GB — enables efficient PIC for large binaries
- LDP/STP (Load/Store Pair): loads or stores two registers in one instruction — doubles memory bandwidth for stack frames

11.3 x86-64 (Intel, 2003 extension of x86/1978) — CISC Heritage

x86-64 retains and extends the full CISC addressing mode heritage originating from the Intel 8086 (1978). Despite its complexity, it remains the dominant desktop and server architecture due to vast ecosystem inertia and the continuous improvements of modern out-of-order microarchitectures that hide much of the encoding complexity from the programmer.

- Full SIB (Scale-Index-Base) mode: EA = Base + Index×{1,2,4,8} + 32-bit displacement — most expressive single mode in any ISA
- RIP-relative (added in x86-64): branches and data references relative to RIP — finally enables true PIC on x86, enabling ASLR on 64-bit Windows/Linux
- Segment registers (FS, GS): used for Thread Local Storage (TLS) — a form of base-register addressing for per-thread data
- LOCK prefix with memory operands: atomic read-modify-write using any addressing mode — critical for OS and synchronization code

12. Conclusion

This case study has examined addressing modes from first principles through hardware implementation, programming practice, performance analysis, and security implications. The twelve modes analyzed represent the complete design space that an ISS architect must navigate when building a new processor.

The central finding is clear: **five modes cover the overwhelming majority of practical programs** — Immediate, Register, Base+Offset, PC-Relative, and Register Indirect. These five modes, implemented with a load-store discipline and a simple AGU (adder only), produce a processor that:

- Executes all fundamental programming constructs (arithmetic, arrays, structs, branches, function calls, recursion)
- Supports Position-Independent Code and ASLR for modern security requirements
- Can be implemented in a clean 5-stage pipeline with a simple decoder and AGU
- Can be formally verified and safely extended with optional mode extensions as workload needs evolve

The RISC-versus-CISC comparison reveals that the choice of addressing modes is not just a hardware decision — it shapes compiler design, ABI conventions, binary format, operating system interfaces, and security architecture. Modern ISAs (RISC-V, AArch64) deliberately converge on the minimal core set, adding complexity only where benchmarking demonstrates measurable benefit.

For the CAO course at VIT Vellore, the key takeaway is that addressing mode design exemplifies the fundamental engineering trade-off in computer architecture: *every feature has a cost*. The discipline of carefully measuring benefit against cost before adding complexity to an instruction set is what separates elegant, long-lived ISAs from bloated, unmaintainable ones.

Final Summary — Key Takeaways for CAO
1. EA computation is the defining hardware difference between addressing modes
2. Five core modes (Imm, Reg, Base+Off, PC-Rel, Reg-Indir) cover all essential programming patterns
3. Load-store discipline separates memory access from computation — RISC's key pipeline advantage
4. PC-Relative is non-negotiable for security: enables ASLR, PIE, and position-independent shared libraries
5. Auto-Increment saves instruction count for sequential access; Scaled Indexed saves instructions for typed arrays
6. CISC gains code density at the cost of decode complexity; RISC gains pipeline efficiency at the cost of instruction count
7. Every additional mode increases AGU/decoder hardware cost, verification burden, and potential security surface

8. Benchmark before adding modes — measure real workloads, not theoretical scenarios